

Assignment 1: ANN Fundamentals - Building from Scratch

Title (AI Strictly prohibited- strict ai and plagiarism policies rejections can't be reversed)

Implementing Artificial Neural Networks from Scratch using NumPy

Problem Statement

Build a multi-layer perceptron (MLP) neural network from scratch using only NumPy (no deep learning frameworks) to classify the Iris flower species. This assignment will help you understand the mathematical foundations of neural networks, including forward propagation, backpropagation, gradient descent, and activation functions.

Dataset

Iris Dataset

- **Source:** [UCI Machine Learning Repository - Iris Dataset](#)
- **Alternative:** Use `sklearn.datasets.load_iris()`
- **Description:** 150 samples, 4 features (sepal length/width, petal length/width), 3 classes (Setosa, Versicolor, Virginica)

Detailed Instructions

Part 1: Data Preparation (15%)

1. Load the Iris dataset and perform exploratory data analysis (EDA)
2. Visualize feature distributions using histograms and pair plots
3. Split the dataset into training (70%), validation (15%), and test (15%) sets
4. Normalize/standardize features using z-score normalization
5. Convert labels to one-hot encoded vectors

Part 2: Neural Network Implementation (50%)

1. **Initialize Network Architecture**
 - Input layer: 4 neurons (features)
 - Hidden layer 1: 8 neurons with sigmoid activation
 - Hidden layer 2: 6 neurons with sigmoid activation
 - Output layer: 3 neurons with softmax activation

2. Implement Core Functions

- `initialize_parameters()`: Random weight initialization with proper scaling
- `sigmoid()` and `sigmoid_derivative()`: Activation functions
- `softmax()`: Output layer activation
- `forward_propagation()`: Compute activations layer by layer
- `compute_loss()`: Cross-entropy loss function
- `backward_propagation()`: Compute gradients using chain rule
- `update_parameters()`: Gradient descent update rule

3. Training Loop

- Implement mini-batch gradient descent (batch size: 16)
- Train for 1000 epochs with learning rate = 0.01
- Track and store training loss and validation accuracy every 50 epochs
- Implement early stopping if validation accuracy plateaus

Part 3: Evaluation and Analysis (20%)

1. Test the trained model on the test set
2. Generate confusion matrix and classification report
3. Visualize decision boundaries (use PCA for 2D projection)
4. Compare performance with sklearn's `MLPClassifier`

Part 4: Experimentation (15%)

1. Experiment with different learning rates (0.001, 0.01, 0.1)
2. Try different activation functions (ReLU, tanh)
3. Test different network architectures (more/fewer layers)
4. Document your findings in a comparison table

Expected Outputs

1. Code Deliverables

- Well-commented Python script (.py) or Jupyter notebook (.ipynb)
- Modular functions with clear docstrings

2. Visualizations

- Training loss curve (loss vs. epochs)
- Training and validation accuracy curves
- Confusion matrix heatmap
- Feature correlation heatmap
- Decision boundary visualization (2D)

3. Performance Metrics

- Test accuracy (target: >90%)
- Precision, recall, F1-score for each class
- Comparison table of different configurations

4. Written Report (2-3 pages)

- Architecture justification
- Mathematical explanation of backpropagation
- Analysis of results and learning curves
- Discussion of hyperparameter effects

Findings/Observations to Document

1. How does learning rate affect convergence speed and final accuracy?
2. Which activation function performs best for this dataset and why?
3. What is the effect of network depth on overfitting?
4. How does batch size impact training stability?
5. Compare your implementation's performance with sklearn's MLPClassifier

Evaluation Rubric (100 points)

Criteria	Points	Description
Code Correctness	40	Proper implementation of forward/backward propagation, correct matrix operations, bug-free execution
Code Quality	10	Clean code, proper commenting, modular design, use of functions
Mathematical Understanding	20	Correct gradient calculations, proper explanation of backpropagation

Criteria	Points	Description
Results & Visualization	15	Accuracy >90%, all required plots generated, clear visualizations
Experimentation	10	Thorough testing of hyperparameters, meaningful comparisons
Report & Analysis	5	Clear explanations, insightful observations, professional presentation

Bonus Points (5)

- Implement L2 regularization
- Add momentum or Adam optimizer
- Create an interactive visualization of learning process
